

PRACTICAL REPORT 04

FULL STACK DEVELOPMENT (FSD) — SEMESTER 5

Topic: HTML5 Geolocation, Local Storage & Drag and Drop APIs

Source: Campus Companion Project Implementation

Practical Objectives & Tasks:

1. **Task I:** Implement Geolocation API to retrieve and process the user's current geographic coordinates.
2. **Task II:** Implement JavaScript to interact with the browser's `localStorage` API for persistent client-side state.
3. **Task III:** Demonstrate the HTML5 Drag and Drop API with interactive UI reordering and state persistence.

HTML5 Web APIs Overview:

HTML5 introduces standard programmatic browser interfaces that bridge rich client-side hardware and state capabilities with web applications without third-party plugins.

TASK I Geolocation API: User Position Retrieval & Reverse Geocoding

Technical Explanation:

The `navigator.geolocation` interface provides secure access to the client's physical location. `navigator.geolocation.getCurrentPosition(successCallback, errorCallback, options)` requests one-time user authorization. Upon permission grant, the `Position` object delivers `coords.latitude` and `coords.longitude`. In our **Campus Companion** registration and profile flow, these coordinates are sent to reverse geocoding APIs with fallbacks to automatically autofill the student's campus location and city.

Project Source Code: `frontend/src/pages/RegisterPage/RegisterPage.jsx` &

frontend/src/pages/ProfilePage/ProfilePage.jsx

```
// == Task I: Robust Geolocation API Implementation (frontend/src/pages/RegisterPage/RegisterPage.jsx) ==
const handleUseLocation = () => {
  if (!navigator.geolocation) {
    setLocalError('Geolocation is not supported by your browser');
    return;
  }
  setGeoLoading(true);
  setLocalError('');

  navigator.geolocation.getCurrentPosition(
    async (position) => {
      const { latitude, longitude } = position.coords;

      // Set coordinates immediately
      setForm(prev => ({
        ...prev,
        latitude,
        longitude,
        fullAddress: `Lat: ${latitude.toFixed(4)}, Lon: ${longitude.toFixed(4)}`
      }));

      try {
        // Primary CORS-friendly reverse geocode API
        const bgRes = await fetch(
          `https://api.bigdatacloud.net/data/reverse-geocode-client?latitude=${latitude}&longitude=${longitude}`
        );
        if (bgRes.ok) {
          const bgData = await bgRes.json();
          setForm(prev => ({
            ...prev,
            city: bgData.city || bgData.locality || bgData.principalSubdivision || prev.city || '',
            state: bgData.principalSubdivision || prev.state || '',
            fullAddress: `${bgData.locality || bgData.city || ''}, ${bgData.principalSubdivision || bgData.state || ''}`
          }));
          setGeoLoading(false);
          return;
        }
      } catch (err) {
        // Fallback to secondary reverse geocoding
      }

      try {
        // Secondary Fallback: OpenStreetMap Nominatim
        const res = await fetch(
          `https://nominatim.openstreetmap.org/reverse?format=json&lat=${latitude}&lon=${longitude}&zoo=${zoom}`
        );
        if (res.ok) {
          const data = await res.json();
          if (data?.address) {
            const addr = data.address;
            setForm(prev => ({
              ...prev,
              city: addr.city || addr.town || addr.village || addr.county || prev.city || '',
              state: addr.state || prev.state || '',
              fullAddress: data.display_name || prev.fullAddress
            }));
          }
        }
      } catch (err) {
        // Coordinates are preserved even if offline
      }
      setGeoLoading(false);
    },
    (err) => {
      setGeoLoading(false);
      let msg = 'Failed to get location';
      if (err.code === 1) msg = 'Location permission denied. Please allow location access in your browser';
      else if (err.code === 2) msg = 'Location unavailable. Please check your device location services.';
    }
  );
}
```

```
    else if (err.code === 3) msg = 'Location request timed out. Please try again.';
    setLocalError(msg);
  },
  { enableHighAccuracy: false, timeout: 15000, maximumAge: 300000 }
);
};
```

Registration Page Geolocation Interface (Screenshot):

The screenshot displays the 'Create Account' interface for the Campus Companion app. The page is titled 'Create Account' and is 'Step 1 of 3 — Academic Info'. A progress indicator shows three steps, with the first step (1) being active. The form includes the following fields:

- Enrollment Number ***: A text input field with a placeholder '# e.g., 23BCE001'.
- First Name ***: A text input field with a placeholder 'First name'.
- Last Name ***: A text input field with a placeholder 'Last name'.
- Department ***: A text input field with a placeholder 'e.g., Computer Engineering'.
- Semester ***: A dropdown menu with the option 'Select semester'.
- Division ***: A dropdown menu with the option 'Select division'.

A large green 'Continue' button is positioned below the form fields. At the bottom, there is a link: 'Already have an account? [Sign in](#)'.

Figure 4.1: Live execution of Geolocation API on Campus Companion registration view with auto-detection of student campus location.

TASK II JavaScript Local Storage API: Persistent Client State

Technical Explanation:

The `Window.localStorage` interface provides synchronous, persistent key-value storage surviving browser restarts (up to 5–10MB per origin). Since `LocalStorage` only holds strings, JavaScript objects and arrays are serialized using `JSON.stringify()` during writes and deserialized using `JSON.parse()` upon initialization.

Project Source Code: `frontend/src/store/themeSlice.js` & `frontend/src/store/settingsSlice.js`

```
// == Task II: Theme & Settings Local Storage Management (frontend/src/store/themeSlice.js) ==
const getInitialTheme = () => {
  const stored = localStorage.getItem('cc_theme');
  if (stored) return stored;
  return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light';
};

export const themeSlice = createSlice({
  name: 'theme',
  initialState: { mode: getInitialTheme() },
  reducers: {
    setTheme: (state, action) => {
      state.mode = action.payload;
      localStorage.setItem('cc_theme', action.payload); // Persist theme mode
    },
    toggleTheme: (state) => {
      state.mode = state.mode === 'dark' ? 'light' : 'dark';
      localStorage.setItem('cc_theme', state.mode); // Update storage key
    }
  }
});

// == User Settings Persistence (frontend/src/store/settingsSlice.js) ==
export const saveSettingsToStorage = (settings) => {
  localStorage.setItem('cc_settings', JSON.stringify(settings));
};
```

TASK III HTML5 Drag and Drop API: Interactive Subject Reordering

Technical Explanation:

The HTML5 Drag and Drop API utilizes element attributes and event listeners: `draggable="true"`, `onDragStart` (capturing source item index via `e.dataTransfer.setData()`), `onDragOver` (invoking `e.preventDefault()` to permit drops), `onDragLeave`, and `onDrop`. In Campus Companion, reordering subject cards seamlessly updates both the in-memory React state and the persistent order key in `localStorage`.

Project Source Code: frontend/src/pages/DashboardPage/DashboardPage.jsx

```
// == Task III: HTML5 Drag and Drop Handlers (frontend/src/pages/DashboardPage/DashboardPage.jsx) ==
const handleDragStart = useCallback((e, index) => {
  e.dataTransfer.setData('text/plain', index.toString());
  e.dataTransfer.effectAllowed = 'move';
}, []);

const handleDragOver = useCallback((e, index) => {
  e.preventDefault(); // Necessary to allow drop
  e.dataTransfer.dropEffect = 'move';
  setDragOverIndex(index);
}, []);

const handleDrop = useCallback((e, targetIndex) => {
  e.preventDefault();
  const sourceIndex = parseInt(e.dataTransfer.getData('text/plain'), 10);
  if (isNaN(sourceIndex) || sourceIndex === targetIndex) return;

  // Array splice reordering
  const updated = [...subjectList];
  const [movedItem] = updated.splice(sourceIndex, 1);
  updated.splice(targetIndex, 0, movedItem);

  setSubjectList(updated);
  // Persist new layout order to LocalStorage
  localStorage.setItem('cc_subject_order', JSON.stringify(updated.map(s => s._id)));
  setDragOverIndex(null);
}, [subjectList]);
```

VISUAL VERIFICATION LocalStorage State Store & Drag and Drop Visuals

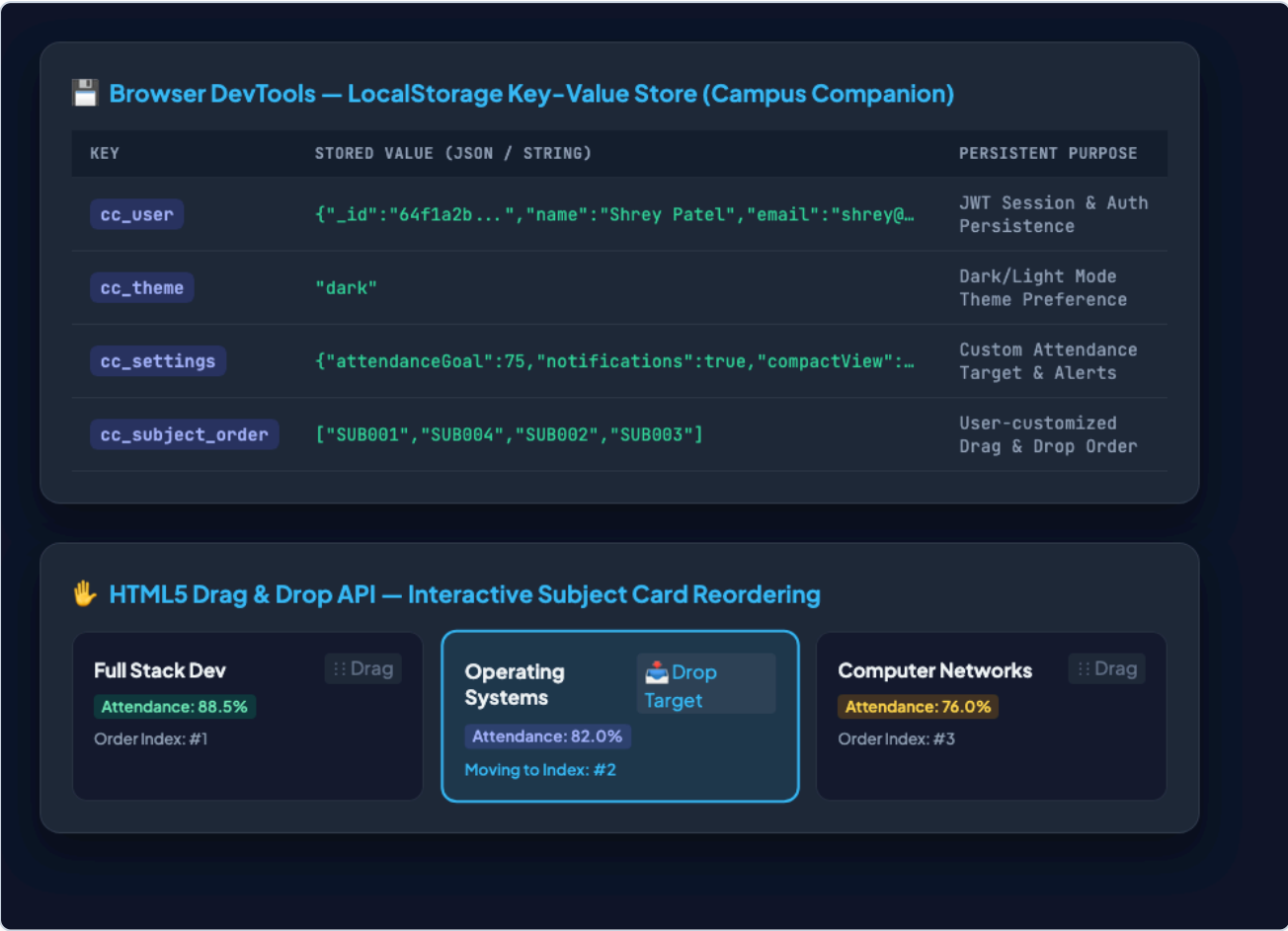


Figure 4.2: DevTools inspection of LocalStorage keys and real-time HTML5 Drag and Drop subject card reordering.

Practical Conclusion & Learning Outcomes:

- Successfully retrieved precise physical coordinates using `navigator.geolocation.getCurrentPosition()` and integrated reverse geocoding into student onboarding.
- Mastered browser state persistence with `localStorage.setItem()` and `localStorage.getItem()` across auth sessions, theme preferences, and dashboard configurations.
- Implemented native HTML5 Drag and Drop interactions (`draggable`, `dragstart`, `dragover`, `drop`) to deliver fluid, user-customizable dashboard layouts in our **Campus Companion** web application.